

Algoritmi e strutture dati

Riccardo Di Michele

1 settembre 2026

Introduzione

L'obiettivo della materia è quello di studiare la **complessità computazionale** degli algoritmi nel tempo; in questo testo non andremo a spiegare le nozioni primitive dell'informatica (esempio: cosa è un algoritmo, una condizione if ecc.). Un esempio di complessità computazionale:

```
1 Algorithm: CalcoloMatriceDistanzeManhattan(A, B, n)
   Input  : Due array/vettori A e B ciascuno di  $n$  numeri reali
   Output: Una matrice C di dimensione  $n \times n$  con le differenze

2 for  $i \leftarrow 1$  to  $n$  do
3   for  $j \leftarrow 1$  to  $n$  do
4      $C[i, j] \leftarrow A[i] \times B[j] + i \times j;$ 
5   end
6 end
```

L'algoritmo appena mostrato ha costo computazionale $T(n) = n^2$, ma cosa vuol dire e soprattutto come usiamo questa informazione a nostro vantaggio? Sapere il costo di un algoritmo ci va in nostro aiuto per vari motivi: se l'algoritmo è troppo costoso potremmo apportare dei cambiamenti, o addirittura scegliere un algoritmo già esistente più efficiente di quello scelto prima, o ancora, per capire quale è l'input più grande che l'algoritmo può "reggere" prima di perdere troppo tempo.

Cominciamo allora a rispondere alla domanda più importante. Definiamo la *complessità computazionale* come una funzione che mappa la dimensione dell'input di un algoritmo al suo "tempo" di calcolo

$$\text{costo} : \text{input} \rightarrow \text{tempo}$$

Adesso dobbiamo però definire questo *input* e questo *tempo*. Il primo lo definiamo usando il **criterio del costo uniforme**, ovvero la dimensione dell'input dipende solo da quanti elementi ha quell'input; se prendiamo

come esempio un array di 20 elementi il suo costo sarà di $n = 20$.
 Definiamo il tempo invece come il numero di operazioni elementari, ovvero operazioni che la macchina riesce a fare senza l'aiuto di altri operazioni.

- $a = a + 1$ **operazione elementare**
- $a = a - 1$ **operazione elementare**
- $swap(A[1], A[2])$ **operazione elementare**
- $sin(a)$ **operazione** considerata al giorno d'oggi **elementare**
- $max(array, a)$ **operazione non elementare**

L'ultima operazione, ovvero la funzione $max()$ non viene considerata elementare poichè all'interno di essa (pure se non detto in maniera esplicita) avremo **sicuramente** più di un'operazione elementare, come per esempio molteplici assegnazioni di variabili.

Con le informazioni acquisite possiamo adesso verificare il costo dell'algoritmo citato prima:

Algorithm 1: CalcoloMatriceDistanzeManhattan(A, B, n)

Input : Due array/vettori A e B ciascuno di n numeri reali
Output: Una matrice C di dimensione $n \times n$ con le differenze

```

1 for  $i \leftarrow 1$  to  $n$  do
2   for  $j \leftarrow 1$  to  $n$  do
3      $C[i, j] \leftarrow A[i] \times B[j] + i \times j$ ;
4   end
5 end
```

Per analizzare il codice cominciamo a partire dalle operazioni più interne, prendiamo l'operazione alla riga 3

$$C[i, j] \leftarrow A[i] \times B[j] + (i \times j)$$

Il costo di questa riga sarebbe pari al numero di operazioni elementari al suo interno, quindi 4; in questo caso diremmo che il **costo esatto** è pari a $R3 = 4$. Andando a ritroso quindi analizziamo ora la parte di codice meno interna: il *ciclo for* che va da R2 a R4. Questo è un ciclo che va da $j \leftarrow 1$ a $j \leftarrow n$, quindi farà sempre n cicli. Col costo della riga R3 possiamo quindi calcolare il costo di questo ciclo come:

$$\sum_{j=1}^n R3 \rightarrow n \cdot R3$$

Con la stessa logica analizziamo infine il ciclo più esterno, quello tra R1 e R5; in questo caso la variabile contatore è i , ma la logica, e il numero di cicli, non cambia affatto.

$$\sum_{i \leftarrow 1}^n \sum_{j \leftarrow 1}^n R3 \rightarrow n \cdot n \cdot R3 \rightarrow n^2 \cdot R3$$

Ma nella maggior parte dei nostri esercizi il costo esatto non ci interessa proprio, ci interessa invece il **costo asintotico**, ovvero il costo dell'algoritmo considerando un input n sempre più grande.

Poichè il costo di $R3$ è un costo *fisso*, non dipendente dalla grandezza dell'input n , nel **calcolo asintotico** diventa un dato insignificante, è per questo che in tanti esercizi lo troviamo scritto come c ; il calcolo di prima diventerà allora:

$$\sum_{i \leftarrow 1}^n \sum_{j \leftarrow 1}^n c \rightarrow n \cdot n \cdot c \rightarrow c \cdot n^2 \rightarrow n^2 = T(n)$$

Per lo stesso motivo viene direttamente omesso il valore c quando raggiungiamo il risultato finale.